CodeQL Alert Triage Report #225

1. CodeQL Alert Info

Rule cpp/uncontrolled-process-operation
Severity WARNING
Rule description Uncontrolled process operation

/magick/delegate.c

```c
  if (message != (char *) NULL)
    *message='\0';
#if defined(MAGICKCORE_POSIX_SUPPORT)
#if !defined(MAGICKCORE_HAVE_EXECVP)
  status=system(sanitize_command);
#else
  if ((asynchronous != MagickFalse) ||
      (strpbrk(sanitize_command,"&;<>|") != (char *) NULL))
    status=system(sanitize_command);
  else
    {
      pid_t
        child_pid;

      /*
        Call application directly rather than from a shell.
      */
```

2. Triage

Verdict REAL
Severity HIGH
Explanation This is a real shell-command injection surface in the delegate execution path. `ExternalDelegateCommand` takes a `command` string, runs it through `SanitizeDelegateCommand`, and then executes it with `system()` whenever the sanitized command is asynchronous or still contains any of `&;<>|` (`magick/delegate.c:395-406`). The sanitizer is not a shell-escaping routine: its whitelist explicitly permits shell metacharacters including `& ; < > | ' " % = ~` and backtick, so attacker-controlled data embedded in the command is not neutralized (`magick/delegate.c:333-342`).

The command string is not fixed text. Delegate commands are loaded from XML configuration (`delegate_info->commands`) and then expanded with `InterpretImageProperties(image_info,image,commands[i])` immediately before execution (`magick/delegate.c:1611-1633`, `magick/delegate.c:1254-1302`). That expansion uses runtime image and option data, which matches the CodeQL sources from file content, environment, and command-line arguments. This means externally influenced values are interpolated into a shell command template and then passed to `system()`.

There is one guard, `IsRightsAuthorized(..., arguments[1])`, but it only authorizes the executable/delegate name extracted from `StringToArgv(command,...)`; it does not validate or quote the interpolated argument values (`magick/delegate.c:368-389`). Therefore it does not prevent shell metacharacter injection in the rest of the command string.

In this codebase, delegate execution is a normal feature used by coders such as PDF/PS/SVG/XPS (`coders/pdf.c`, `coders/ps.c`, `coders/svg.c`, `coders/xps.c` call `ExternalDelegateCommand`). The

issue is not merely that ImageMagick launches external tools; the unsafe part is that untrusted runtime data is inserted into a shell command while preserving shell syntax and then dispatched through `system()`.

3. Analysis

3.1. CodeQL's Hypothesis

CodeQL flagged this because `ExternalDelegateCommand` ultimately passes a dynamically built delegate command string to `system()`, and the string appears to incorporate untrusted data from file contents, environment variables, and command-line/image properties. The suspicious point is whether the local sanitizer or delegate policy meaningfully neutralizes shell injection before execution.

3.2. Hypothesis Validation

The alert hypothesis that untrusted text reaches a process operation is supported: the sink is a real `system()` call, and the pre-processing step is only character filtering, not safe argument escaping.`system(sanitize_command)` is called when the command is asynchronous or contains shell metacharacters.

magick/delegate.c

```c
    {
      (void) FormatLocaleFile(stderr,"%s\n",command);
      (void) fflush(stderr);
    }
  sanitize_command=SanitizeDelegateCommand(command);
  if (asynchronous != MagickFalse)
    (void) ConcatenateMagickString(sanitize_command,"&",MaxTextExtent);
  if (message != (char *) NULL)
    *message='\0';
#if defined(MAGICKCORE_POSIX_SUPPORT)
#if !defined(MAGICKCORE_HAVE_EXECVP)
  status=system(sanitize_command);
#else
  if ((asynchronous != MagickFalse) ||
      (strpbrk(sanitize_command,"&;<>|") != (char *) NULL))
    status=system(sanitize_command);
  else
    {
      pid_t
        child_pid;
```

The sanitizer whitelist keeps `&;<>|`, quotes, and backtick instead of shell-escaping them.

magick/delegate.c

```c

  register char
    *p;

  static char
    whitelist[] =
      "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789_- "
      ".@&;<>()|/\\\'\":%=~`";

  sanitize_command=AcquireString(command);
```

```c
  p=sanitize_command;
  q=sanitize_command+strlen(sanitize_command);
  for (p+=strspn(p,whitelist); p != q; p+=strspn(p,whitelist))
    *p='_';
  return(sanitize_command);
}

MagickExport int ExternalDelegateCommand(const MagickBooleanType asynchronous,
```

The executed string is dynamically constructed from configuration templates plus runtime image properties, so the command content is not fixed and can include externally influenced substitutions.Delegate command templates are expanded with `InterpretImageProperties(image_info,image,commands[i])` and then executed.

magick/delegate.c

```c
        break;
      }
  }
status=MagickFalse;
command=InterpretImageProperties(image_info,image,commands[i]);
if (command != (char *) NULL)
  {
    /*
      Execute delegate.
    */
    status=ExternalDelegateCommand(delegate_info->spawn,image_info->verbose,
      command,(char *) NULL,exception) != 0 ? MagickTrue : MagickFalse;
    if (delegate_info->spawn != MagickFalse)
      {
        ssize_t
          count;
```

Delegate command templates come from XML `command` attributes loaded into `delegate_info->commands`.

magick/delegate.c

```c
    {
      case 'C':
      case 'c':
      {
        if (LocaleCompare((char *) keyword,"command") == 0)
          {
            char
              *commands;

            commands=AcquireString(token);
#if defined(MAGICKCORE_WINDOWS_SUPPORT)
            if (strchr(commands,'@') != (char *) NULL)
              {
                char
                  path[MaxTextExtent];

                NTGhostscriptEXE(path,MaxTextExtent);
                (void) SubstituteString((char **) &commands,"@PSDelegate@",
```

```c
                  path);
                (void) SubstituteString((char **) &commands,"\\","/");
              }
#endif
            (void) SubstituteString((char **) &commands,"&amp;","&");
            (void) SubstituteString((char **) &commands,"&quot;","\"");
            (void) SubstituteString((char **) &commands,"&gt;",">");
            (void) SubstituteString((char **) &commands,"&lt;","<");
            delegate_info->commands=commands;
            break;
          }
        break;
      }
```

The main built-in guard does not defeat injection: it authorizes which delegate program may run, but does not constrain attacker-controlled argument fragments that remain in the shell command.Authorization checks only the delegate executable token `arguments[1]` derived from the command string.

magick/delegate.c

```c
register ssize_t
  i;

status=(-1);
arguments=StringToArgv(command,&number_arguments);
if (arguments == (char **) NULL)
  return(status);
if (*arguments[1] == '\0')
  {
    for (i=0; i < (ssize_t) number_arguments; i++)
      arguments[i]=DestroyString(arguments[i]);
    arguments=(char **) RelinquishMagickMemory(arguments);
    return(-1);
  }
rights=ExecutePolicyRights;
domain=DelegatePolicyDomain;
if (IsRightsAuthorized(domain,rights,arguments[1]) == MagickFalse)
  {
    errno=EPERM;
    (void) ThrowMagickException(exception,GetMagickModule(),PolicyError,
      "NotAuthorized","`%s'",arguments[1]);
    for (i=0; i < (ssize_t) number_arguments; i++)
      arguments[i]=DestroyString(arguments[i]);
    arguments=(char **) RelinquishMagickMemory(arguments);
    return(-1);
  }
if (verbose != MagickFalse)
  {
    (void) FormatLocaleFile(stderr,"%s\n",command);
```

`StringToArgv` tokenizes the command string but does not provide shell-safe quoting for subsequent `system()` use.

magick/string.c

```
%    o argc:  This integer pointer returns the number of arguments in the
%      list.
%
*/
MagickExport char **StringToArgv(const char *text,int *argc)
{
  char
    **argv;

  register const char
    *p,
    *q;

  register ssize_t
    i;

  *argc=0;
  if (text == (char *) NULL)
    return((char **) NULL);
  /*
    Determine the number of arguments.
  */
  for (p=text; *p != '\0'; )
  {
    while (isspace((int) ((unsigned char) *p)) != 0)
      p++;
    if (*p == '\0')
      break;
    (*argc)++;
    if (*p == '"')
      for (p++; (*p != '"') && (*p != '\0'); p++) ;
    if (*p == '\'')
      for (p++; (*p != '\'') && (*p != '\0'); p++) ;
    while ((isspace((int) ((unsigned char) *p)) == 0) && (*p != '\0'))
      p++;
  }
  (*argc)++;
  argv=(char **) AcquireQuantumMemory((size_t) (*argc+1UL),sizeof(*argv));
  if (argv == (char **) NULL)
    ThrowFatalException(ResourceLimitFatalError,"UnableToConvertStringToARGV");
  /*
    Convert string to an ASCII list.
  */
  argv[0]=AcquireString("magick");
  p=text;
  for (i=1; i < (ssize_t) *argc; i++)
  {
    while (isspace((int) ((unsigned char) *p)) != 0)
      p++;
    q=p;
    if (*q == '"')
      {
        p++;
        for (q++; (*q != '"') && (*q != '\0'); q++) ;
      }
```

```c
    else
      if (*q == '\'')
        {
          p++;
          for (q++; (*q != '\'') && (*q != '\0'); q++) ;
        }
      else
        while ((isspace((int) ((unsigned char) *q)) == 0) && (*q != '\0'))
          q++;
    argv[i]=(char *) AcquireQuantumMemory((size_t) (q-p)+MaxTextExtent,
      sizeof(**argv));
    if (argv[i] == (char *) NULL)
      {
        for (i--; i >= 0; i--)
```

The path is reachable in normal operation for common formats that require delegates, so this is not dead code or a purely theoretical sink.Delegate execution happens in normal image-processing flow.

magick/delegate.c

```c
command=(char *) NULL;
status=MagickFalse;
(void) CopyMagickString(output_filename,image_info->filename,MaxTextExtent);
(void) CopyMagickString(input_filename,image->filename,MaxTextExtent);
for (i=0; commands[i] != (char *) NULL; i++)
{
  status=AcquireUniqueSymbolicLink(output_filename,image_info->filename);
  if (AcquireUniqueFilename(image_info->unique) == MagickFalse)
    {
      ThrowFileException(exception,FileOpenError,
        "UnableToCreateTemporaryFile",image_info->unique);
      break;
    }
  if (AcquireUniqueFilename(image_info->zero) == MagickFalse)
    {
      (void) RelinquishUniqueFileResource(image_info->unique);
      ThrowFileException(exception,FileOpenError,
        "UnableToCreateTemporaryFile",image_info->zero);
      break;
    }
  if (LocaleCompare(decode,"SCAN") != 0)
    {
      status=AcquireUniqueSymbolicLink(input_filename,image->filename);
      if (status == MagickFalse)
        {
          ThrowFileException(exception,FileOpenError,
            "UnableToCreateTemporaryFile",input_filename);
          break;
        }
    }
  status=MagickFalse;
  command=InterpretImageProperties(image_info,image,commands[i]);
  if (command != (char *) NULL)
    {
      /*
        Execute delegate.
```

```
      */
      status=ExternalDelegateCommand(delegate_info->spawn,image_info->verbose,
        command,(char *) NULL,exception) != 0 ? MagickTrue : MagickFalse;
      if (delegate_info->spawn != MagickFalse)
        {
          ssize_t
            count;
```

A PDF coder path invokes `ExternalDelegateCommand`.

coders/pdf.c

```c
    }

#define ExecuteGhostscriptCommand(command,status) \
{ \
  status=ExternalDelegateCommand(MagickFalse,verbose,command,message, \
    exception); \
  if (status == 0) \
    return(MagickTrue); \
  if (status < 0) \
```

An SVG coder path invokes `ExternalDelegateCommand`.

coders/svg.c

```c
(void) FormatLocaleString(command,MaxTextExtent,GetDelegateCommands(
  delegate_info),input_filename,output_filename,density,background,
  opacity,unique);
density=DestroyString(density);
status=ExternalDelegateCommand(MagickFalse,image_info->verbose,
  command,(char *) NULL,exception);
(void) RelinquishUniqueFileResource(unique);
(void) RelinquishUniqueFileResource(input_filename);
if ((status == 0) && (stat(output_filename,&attributes) == 0) &&
```

Default repository policy does not block delegates globally, so execution is not practically prevented by configuration in this codebase snapshot.The shipped policy file does not disable delegate execution by default; it only contains a cache shared-secret entry.

config/policy.xml

```xml
  with SI prefixes (.e.g 100MB).  In addition, resource policies are maximums
  for each instance of ImageMagick (e.g. policy memory limit 1GB, -limit 2GB
  exceeds policy maximum so memory limit is 1GB).
-->
<policymap>
  <!-- <policy domain="resource" name="temporary-path" value="/tmp"/> -->
  <!-- <policy domain="resource" name="memory" value="2GiB"/> -->
  <!-- <policy domain="resource" name="map" value="4GiB"/> -->
  <!-- <policy domain="resource" name="width" value="10MP"/> -->
  <!-- <policy domain="resource" name="height" value="10MP"/> -->
  <!-- <policy domain="resource" name="area" value="1GB"/> -->
  <!-- <policy domain="resource" name="disk" value="16EB"/> -->
  <!-- <policy domain="resource" name="file" value="768"/> -->
  <!-- <policy domain="resource" name="thread" value="4"/> -->
  <!-- <policy domain="resource" name="throttle" value="0"/> -->
```

```
  <!-- <policy domain="resource" name="time" value="3600"/> -->
  <!-- <policy domain="system" name="precision" value="6"/> -->
  <policy domain="cache" name="shared-secret" value="passphrase"/>
</policymap>

```

3.3. Real-world Triggerability

High. Delegate execution is part of normal processing for several formats, and the code preserves shell metacharacters in expanded command strings before passing them to `system()`. An attacker only needs a workflow that processes a delegate-backed format while controlling an interpolated property such as a filename or related runtime value.

3.3.1. Trigger Path

Delegate command templates are loaded from XML configuration into `delegate_info->commands`.

magick/delegate.c

```c
    {
      case 'C':
      case 'c':
      {
        if (LocaleCompare((char *) keyword,"command") == 0)
          {
            char
              *commands;

            commands=AcquireString(token);
#if defined(MAGICKCORE_WINDOWS_SUPPORT)
            if (strchr(commands,'@') != (char *) NULL)
              {
                char
                  path[MaxTextExtent];

                NTGhostscriptEXE(path,MaxTextExtent);
                (void) SubstituteString((char **) &commands,"@PSDelegate@",
                  path);
                (void) SubstituteString((char **) &commands,"\\","/");
              }
#endif
            (void) SubstituteString((char **) &commands,"&amp;","&");
            (void) SubstituteString((char **) &commands,"&quot;","\"");
            (void) SubstituteString((char **) &commands,"&gt;",">");
            (void) SubstituteString((char **) &commands,"&lt;","<");
            delegate_info->commands=commands;
            break;
          }
        break;
      }
```

At runtime, `InterpretImageProperties(image_info,image,commands[i])` expands the template using image and option data that can be influenced by external input.

magick/delegate.c

```c
        break;
      }
```

```c
  }
status=MagickFalse;
command=InterpretImageProperties(image_info,image,commands[i]);
if (command != (char *) NULL)
  {
    /*
      Execute delegate.
    */
```

The expanded command is only weakly sanitized and is executed with `system()` when shell metacharacters are present or asynchronous execution is requested.

magick/delegate.c

```c
    {
      (void) FormatLocaleFile(stderr,"%s\n",command);
      (void) fflush(stderr);
    }
  sanitize_command=SanitizeDelegateCommand(command);
  if (asynchronous != MagickFalse)
    (void) ConcatenateMagickString(sanitize_command,"&",MaxTextExtent);
  if (message != (char *) NULL)
    *message='\0';
#if defined(MAGICKCORE_POSIX_SUPPORT)
#if !defined(MAGICKCORE_HAVE_EXECVP)
  status=system(sanitize_command);
#else
  if ((asynchronous != MagickFalse) ||
      (strpbrk(sanitize_command,"&;<>|") != (char *) NULL))
    status=system(sanitize_command);
  else
    {
      pid_t
        child_pid;
```

Common delegate-driven coders such as PDF processing call into this delegate execution path.

coders/pdf.c

```c
    }

#define ExecuteGhostscriptCommand(command,status) \
{ \
  status=ExternalDelegateCommand(MagickFalse,verbose,command,message, \
    exception); \
  if (status == 0) \
    return(MagickTrue); \
  if (status < 0) \
```

4. Impact

An attacker who can control data expanded into a delegate command template, such as filenames, image properties, environment-influenced properties, or command-line supplied values that feed `InterpretImageProperties`, can inject shell syntax and make the process execute unintended commands with the privileges of the ImageMagick process. The practical impact is command execution in delegate-enabled workflows, commonly affecting server-side image conversion pipelines.

5. Remediation

Avoid invoking the shell for delegate commands built from runtime data. Prefer `execve`/`posix_spawn` with a fully separated argv array for all delegate executions, including cases that currently fall back to `system()`. If shell use is unavoidable for specific legacy delegates, apply strict context-appropriate quoting/escaping to every substituted property and reduce the allowed substitution set so filenames and metadata cannot introduce shell syntax. Consider disabling unneeded delegates in policy and rejecting unsafe characters in values interpolated into delegate templates.